Karnataka Residential Educational Institutions Society

ICT Class 10 - Formative Assessment 1

Formative Assessment (FA-1) - Answer Key

Class: Class 10

Assessment Type: Formative Assessment (FA-1)

Total Marks: 30

Duration: 90 minutes

Topics Covered: Networking Concepts, Internet and Web Technologies

Answer Key - Grading Guidelines

Note to Teacher: This answer key provides expected answers and marking guidelines. Accept equivalent answers where appropriate. Partial marks may be awarded for incomplete but correct responses.

Part A: Written Concept Paper (20 Marks)

Section I: MCQ (5 x 1 = 5 marks)

1. **Answer:** b - Transmission Control Protocol/Internet Protocol
   - **Explanation:** TCP/IP stands for Transmission Control Protocol/Internet Protocol, which is the standard communication protocol suite for the Internet.
2. **Answer:** c - Transmission Control Protocol/Internet Protocol
   - **Explanation:** The full form is Transmission Control Protocol/Internet Protocol. It governs how data is transmitted across networks.
3. **Answer:** c - Router
   - **Explanation:** A router connects multiple networks and directs data packets between them based on IP addresses.
4. **Answer:** b - Educational institutions
   - **Explanation:** The `.edu` top-level domain is reserved for educational institutions such as universities and schools.
5. **Answer:** a - 2048
   - **Explanation:** The practical maximum length of a URL is approximately 2048 characters as per standard browser limitations.

Section II: Short Answer (5 x 2 = 10 marks)

1. **Differentiate between LAN and WAN**
   - **Answer:** LAN (Local Area Network) covers a small geographical area like a school or office building. WAN (Wide Area Network) covers a large geographical area like a city or country. LAN examples: school computer lab network, office home network. WAN examples: Internet, bank network connecting branches across cities.
   - **Marking:** 1 mark for each correct differentiation point, 0.5 marks for each example.
2. **Explain the client-server model**
   - **Answer:** In the client-server model, clients (computers) request services from a central server. The server processes requests and returns responses. Diagram should show multiple clients connected to a central server. Example: When you open a website, your browser (client) requests a webpage from the web server.
   - **Marking:** 1 mark for diagram, 1 mark for explanation with example.
3. **Role of DNS server and domain name resolution**
   - **Answer:** DNS (Domain Name System) translates human-readable domain names (www.example.com) into IP addresses. When you type a URL, the browser contacts a DNS server which looks up the IP address. The DNS server checks its cache, then queries root servers, TLD servers, and authoritative servers to resolve the name.
   - **Marking:** 0.5 marks for DNS purpose, 1 mark for resolution process, 0.5 marks for example.

4. **Compare HTTP and HTTPS**
   - **Answer:** HTTP (HyperText Transfer Protocol) transmits data in plain text. HTTPS (HyperText Transfer Protocol Secure) encrypts data using SSL/TLS. HTTPS is preferred for banking because it ensures data confidentiality, integrity, and authentication. It prevents eavesdropping and man-in-the-middle attacks.
   - **Marking:** 1 mark for comparison, 1 mark for banking preference explanation.

5. **What is cloud computing? List service models**
   - **Answer:** Cloud computing delivers computing services (servers, storage, databases, networking) over the Internet on a pay-as-you-go basis. Service models: IaaS (Infrastructure as a Service) provides virtual machines and storage; PaaS (Platform as a Service) provides development platforms; SaaS (Software as a Service) provides ready-to-use applications.
   - **Marking:** 1 mark for definition, 0.5 marks for each correct service model.

Section III: Long Answer (5 x 1 = 5 marks)

1. **TCP/IP protocol suite**
   - **Answer:** The TCP/IP model has 4 layers: (1) Application Layer - HTTP, FTP, SMTP, DNS protocols handle user applications; (2) Transport Layer - TCP and UDP ensure reliable/unreliable data delivery between hosts; (3) Internet Layer - IP protocol handles addressing and routing of packets; (4) Network Access Layer - Ethernet and Wi-Fi handle physical transmission. Diagram should show all 4 layers with protocols at each level.
   - **Marking:** 1 mark for correct diagram, 1 mark for each layer description with protocol (up to 3 marks).

Part B: Hands-on Practical Lab Task (5 marks)

Program: CSV Student Data Processing

```
import csv

# Read CSV and process student data
students = []
with open('students.csv', 'r') as f:
    reader = csv.DictReader(f)
    for row in reader:
        name = row['Name']
        roll = row['Roll']
        marks = [int(row['Sub1']), int(row['Sub2']), int(row['Sub3'])]
        total = sum(marks)
        percentage = total / 3

        if percentage >= 90:
            grade = 'A'
        elif percentage >= 75:
            grade = 'B'
        elif percentage >= 60:
            grade = 'C'
        else:
            grade = 'D'

        students.append({
            'Name': name, 'Roll': roll, 'Total': total,
            'Percentage': percentage, 'Grade': grade
        })
```

```
# Write results to new file
with open('results.csv', 'w', newline='') as f:
    writer = csv.DictWriter(f, fieldnames=['Name', 'Roll', 'Total', 'Percentage',
'Grade'])
    writer.writeheader()
    writer.writerows(students)

# Find topper
topper = max(students, key=lambda x: x['Percentage'])
print(f"Class Topper: {topper['Name']} with {topper['Percentage']:.2f}%")
```

- **Marking:** 1 mark for CSV reading, 1 mark for grade calculation, 1 mark for file writing, 1 mark for topper finding, 1 mark for code structure.

Part C: Notebook Maintenance (5 marks)

1. Neatness and organisation of notebook (2 marks)
2. Completion of all assigned classwork and homework (2 marks)
3. Proper headings, dates, and indexing (1 mark)

Karnataka Residential Educational Institutions Society

ICT Class 10 - Formative Assessment 2

Formative Assessment (FA-2) - Answer Key

Class: Class 10

Assessment Type: Formative Assessment (FA-2)

Total Marks: 30

Duration: 90 minutes

Topics Covered: HTML Fundamentals, CSS Styling

Answer Key - Grading Guidelines

Note to Teacher: This answer key provides expected answers and marking guidelines. Accept equivalent answers where appropriate. Partial marks may be awarded for incomplete but correct responses.

Part A: Written Concept Paper (20 Marks)

Section I: MCQ (5 x 1 = 5 marks)

1. **Answer:** b - `<ol>`
   - **Explanation:** The `<ol>` tag creates an ordered (numbered) list. `<ul>` creates an unordered list, `<li>` defines list items.
2. **Answer:** c - `color`
   - **Explanation:** The CSS `color` property changes the text colour. There is no `font-color` or `text-color` property in CSS.
3. **Answer:** b - `#main`
   - **Explanation:** In CSS, the `#` symbol selects an element by its id attribute. The `.` symbol is for class selectors.
4. **Answer:** a - `src`
   - **Explanation:** The `src` attribute in the `<img>` tag specifies the path to the image source file.
5. **Answer:** b - `padding`
   - **Explanation:** Padding creates space between the element's border and its content. Margin creates space outside the border.

Section II: Short Answer (5 x 2 = 10 marks)

1. **HTML Table Code**
   - **Answer:**

```html
<table border="1">
  <tr>
    <th>Name</th>
    <th>Subject</th>
    <th>Marks</th>
  </tr>
  <tr>
    <td>Rahul</td>
    <td>Mathematics</td>
    <td>85</td>
  </tr>
  <tr>
    <td>Priya</td>
    <td>Science</td>
    <td>92</td>
  </tr>
  <tr>
    <td>Amit</td>
    <td>English</td>
    <td>78</td>
```

```
  </tr>
</table>
```

- **Marking:** 1 mark for correct table structure, 1 mark for proper tags.

2. **Block-level vs Inline elements**
   - **Answer:** Block-level elements start on a new line and take full width. Examples: `<div>`, `<p>`, `<h1>`. Inline elements do not start on a new line and only take necessary width. Examples: `<span>`, `<a>`, `<img>`.
   - **Marking:** 1 mark for each correct type with examples.
3. **CSS Box Model**
   - **Answer:** The box model consists of: Content (actual content area), Padding (space between content and border), Border (edge around padding), Margin (space outside border). Diagram should show nested boxes with labels. Each layer adds to the total element size.
   - **Marking:** 0.5 marks for each component, 0.5 marks for diagram.
4. **Navigation Bar CSS**
   - **Answer:**

```css
nav {
  display: flex;
  background-color: #003366;
  padding: 10px;
}
nav a {
  color: white;
  text-decoration: none;
  padding: 10px 20px;
}
nav a:hover {
  text-decoration: underline;
  background-color: #004080;
}
```

- **Marking:** 1 mark for flexbox layout, 1 mark for styling and hover effect.

5. **Class vs ID selector**
   - **Answer:** Class selector (`.classname`) can be used on multiple elements for shared styling. ID selector (`#idname`) is unique to one element. Use classes for reusable styles, IDs for unique elements or JavaScript targeting.
   - **Marking:** 1 mark for difference, 1 mark for use cases.

Section III: Long Answer (5 x 1 = 5 marks)

1. **School Library Management System Homepage**
   - **Answer:** Complete HTML page should include: Header with school name and nav menu using semantic tags; Search form with input fields for title, author, and category dropdown; Table with 5 book entries; CSS styling with colours, borders, and responsive design; Footer with copyright. Key elements: proper HTML5 structure, semantic tags, form validation attributes, responsive media queries.
   - **Marking:** 1 mark for structure, 1 mark for form, 1 mark for table, 1 mark for CSS, 1 mark for responsiveness.

Part B: Hands-on Practical Lab Task (5 marks)

Personal Portfolio Website

Key components:

- Hero section with flexbox layout, background colour, and centered text
- About section with bio paragraph
- Skills section with progress bars using CSS width percentages
- Contact form with proper labels and input types
- Media queries for responsive breakpoints (768px, 480px)
- **Marking:** 1 mark for hero section, 1 mark for about/skills, 1 mark for contact form, 1 mark for responsive design, 1 mark for styling.

Part C: Notebook Maintenance (5 marks)

1. Neatness and organisation of notebook (2 marks)
2. Completion of all assigned classwork and homework (2 marks)
3. Proper headings, dates, and indexing (1 mark)

Karnataka Residential Educational Institutions Society

ICT Class 10 - Formative Assessment 3

Formative Assessment (FA-3) - Answer Key

Class: Class 10

Assessment Type: Formative Assessment (FA-3)

Total Marks: 30

Duration: 90 minutes

Topics Covered: Python Programming - Part 1

Answer Key - Grading Guidelines

Note to Teacher: This answer key provides expected answers and marking guidelines. Accept equivalent answers where appropriate. Partial marks may be awarded for incomplete but correct responses.

Part A: Written Concept Paper (20 Marks)

Section I: MCQ (5 x 1 = 5 marks)

1. **Answer:** a - `import module`
   - **Explanation:** Python uses the `import` keyword to import modules. `include`, `require`, and `using` are not valid Python syntax.
2. **Answer:** b - `<class 'list'>`
   - **Explanation:** `[]` creates an empty list in Python. The `type()` function returns `<class 'list'>`.
3. **Answer:** a - `readline()`
   - **Explanation:** `readline()` reads one line from a file. There is no `getLine()`, `fetchLine()`, or `line()` method.
4. **Answer:** a - Opens file and closes it automatically after the block
   - **Explanation:** The `with` statement ensures the file is properly closed after the block, even if an exception occurs.
5. **Answer:** b - `ZeroDivisionError`
   - **Explanation:** Python raises `ZeroDivisionError` when attempting to divide by zero.

Section II: Short Answer (5 x 2 = 10 marks)

1. **Output and explanation of filter code**
   - **Answer:** Output: `[30, 40, 50]`. The `filter()` function applies the lambda to each element. `lambda x: x > 25` returns True for elements greater than 25. Only matching elements are included in the result list.
   - **Marking:** 1 mark for correct output, 1 mark for explanation.
2. **Differentiate read(), readline(), readlines()**
   - **Answer:** `read()` reads the entire file as a single string. `readline()` reads one line at a time. `readlines()` reads all lines and returns a list of strings. Use `read()` for small files, `readline()` for line-by-line processing, `readlines()` when you need a list.
   - **Marking:** 0.5 marks for each method, 0.5 marks for use cases.
3. **Character frequency function**
   - **Answer:**

```python
def char_frequency(s):
    freq = {}
    for char in s:
        freq[char] = freq.get(char, 0) + 1
    return freq

# Test
print(char_frequency("hello"))  # {'h': 1, 'e': 1, 'l': 2, 'o': 1}
```

- **Marking:** 1 mark for function definition, 1 mark for correct logic.

4. **Try-except for file handling**
   - **Answer:** `try-except` blocks handle runtime errors gracefully. Example:

```
try:
    f = open("data.txt", "r")
    content = f.read()
    f.close()
except FileNotFoundError:
    print("File not found!")
except PermissionError:
    print("No permission to read file!")
```

- **Marking:** 1 mark for purpose, 1 mark for example.

5. **Syntax error vs Runtime error**
   - **Answer:** Syntax errors occur when code violates Python grammar (detected before execution). Example: `print("hello"` (missing parenthesis). Runtime errors occur during execution. Example: `10 / 0` causes ZeroDivisionError.
   - **Marking:** 1 mark for each type with example.

Section III: Long Answer (5 x 1 = 5 marks)

1. **Student Records Management Program**
   - **Answer:**

```
# Write student records
students = [
    {"name": "Rahul", "roll": 1, "marks": 85},
    {"name": "Priya", "roll": 2, "marks": 92},
    {"name": "Amit", "roll": 3, "marks": 78},
    {"name": "Sneha", "roll": 4, "marks": 88},
    {"name": "Vijay", "roll": 5, "marks": 71}
]

with open("students.txt", "w") as f:
    for s in students:
        f.write(f"{s['name']},{s['roll']},{s['marks']}\n")

# Read and display sorted by marks
with open("students.txt", "r") as f:
    lines = f.readlines()

data = []
for line in lines:
    name, roll, marks = line.strip().split(",")
    data.append({"name": name, "roll": int(roll), "marks": int(marks)})

data.sort(key=lambda x: x["marks"], reverse=True)

print("Students sorted by marks:")
for s in data:
    print(f"{s['name']} - Roll: {s['roll']} - Marks: {s['marks']}")

# Find highest marks
topper = max(data, key=lambda x: x["marks"])
print(f"\nTopper: {topper['name']} with {topper['marks']} marks")
```

```
# Count above 75%
count = sum(1 for s in data if s["marks"] > 75)
print(f"Students above 75%: {count}")
```

- **Marking:** 1 mark for file writing, 1 mark for reading, 1 mark for sorting, 1 mark for topper, 1 mark for counting.

Part B: Hands-on Practical Lab Task (5 marks)

File Statistics Program

```
import collections

def analyze_file(filename):
    try:
        with open(filename, 'r') as f:
            content = f.read()

        lines = content.split('\n')
        words = content.split()
        chars = len(content)
        unique_words = set(words)

        # Word frequency
        word_count = collections.Counter(words)
        top5 = word_count.most_common(5)

        # Write summary
        with open('summary.txt', 'w') as f:
            f.write(f"Total Lines: {len(lines)}\n")
            f.write(f"Total Words: {len(words)}\n")
            f.write(f"Total Characters: {chars}\n")
            f.write(f"Unique Words: {len(unique_words)}\n")
            f.write(f"Top 5 words: {top5}\n")

        print(f"Lines: {len(lines)}, Words: {len(words)}, Chars: {chars}")
        print(f"Top 5: {top5}")

    except FileNotFoundError:
        print("File not found!")
    except Exception as e:
        print(f"Error: {e}")

analyze_file("input.txt")
```

- **Marking:** 1 mark for file reading, 1 mark for counting, 1 mark for frequency analysis, 1 mark for file writing, 1 mark for exception handling.

Part C: Notebook Maintenance (5 marks)

1. Neatness and organisation of notebook (2 marks)
2. Completion of all assigned classwork and homework (2 marks)
3. Proper headings, dates, and indexing (1 mark)

Karnataka Residential Educational Institutions Society

ICT Class 10 - Formative Assessment 4

Formative Assessment (FA-4) - Answer Key

Class: Class 10

Assessment Type: Formative Assessment (FA-4)

Total Marks: 30

Duration: 90 minutes

Topics Covered: SQL Database Management

Answer Key - Grading Guidelines

Note to Teacher: This answer key provides expected answers and marking guidelines. Accept equivalent answers where appropriate. Partial marks may be awarded for incomplete but correct responses.

Part A: Written Concept Paper (20 Marks)

Section I: MCQ (5 x 1 = 5 marks)

1. **Answer:** c - `SELECT`
   - **Explanation:** The `SELECT` command retrieves data from database tables. `GET`, `RETRIEVE`, and `FETCH` are not SQL commands.
2. **Answer:** b - `SELECT`, `UPDATE`, `DELETE`
   - **Explanation:** The `WHERE` clause is used with DML commands to filter rows. It is not used with `INSERT` or `CREATE`.
3. **Answer:** b - `SUM()`
   - **Explanation:** `SUM()` returns the total sum of a numeric column. `COUNT()` counts rows, `TOTAL()` and `ADD()` are not aggregate functions.
4. **Answer:** c - `DROP TABLE`
   - **Explanation:** `DROP TABLE` completely removes a table and its structure. `DELETE` removes rows, `TRUNCATE` removes all rows but keeps structure.
5. **Answer:** c - Group rows with same values for aggregate functions
   - **Explanation:** `GROUP BY` groups rows sharing the same values in specified columns for use with aggregate functions.

Section II: Short Answer (5 x 2 = 10 marks)

1. **SQL Queries for Students table**
   - **Answer:**

```sql
-- Students of class 10 with marks above 80
SELECT * FROM Students WHERE class = '10' AND marks > 80;

-- Count students in each section
SELECT section, COUNT(*) FROM Students GROUP BY section;

-- Maximum and minimum marks
SELECT MAX(marks), MIN(marks) FROM Students;

-- Alphabetical order of names
SELECT * FROM Students ORDER BY name ASC;
```

   - **Marking:** 0.5 marks for each correct query.
2. **WHERE vs HAVING clause**
   - **Answer:** `WHERE` filters rows before grouping (used with SELECT, UPDATE, DELETE). `HAVING` filters groups after GROUP BY (used with aggregate functions). Example: `WHERE marks > 80` filters individual rows; `HAVING COUNT(*) > 5` filters groups.
   - **Marking:** 1 mark for difference, 1 mark for examples.

3. **Primary key**
   - **Answer:** A primary key uniquely identifies each row in a table. It cannot contain NULL values and must be unique. A table can have only one primary key (though it can be composite - multiple columns). Example: `roll_no` in Students table.
   - **Marking:** 1 mark for definition, 1 mark for rules.

4. **SQL Output**
   - **Answer:**

```
SELECT UPPER('hello world');      -- 'HELLO WORLD'
SELECT LENGTH('Karnataka');       -- 9
SELECT CONCAT('ICT', ' Class', ' 10');  -- 'ICT Class 10'
```

- **Marking:** 0.5 marks for each correct output.

5. **ACID properties**
   - **Answer:** Atomicity - transaction is all-or-nothing (bank transfer debits and credits). Consistency - database remains valid (constraints always satisfied). Isolation - concurrent transactions don't interfere (two users buying same item). Durability - committed data persists (survives system crash).
   - **Marking:** 0.5 marks for each property with example.

Section III: Long Answer (5 x 1 = 5 marks)

1. **Library Management System Queries**
   - **Answer:**

```
-- (a) Books with price above 500 sorted by title
SELECT * FROM Books WHERE price > 500 ORDER BY title;

-- (b) Members who have not returned any book
SELECT m.name FROM Members m
WHERE m.member_id NOT IN (SELECT member_id FROM Issue WHERE return_date IS NULL);

-- (c) Count how many times each book has been issued
SELECT b.title, COUNT(i.issue_id) AS times_issued
FROM Books b LEFT JOIN Issue i ON b.book_id = i.book_id
GROUP BY b.book_id, b.title;

-- (d) Total revenue from all books
SELECT SUM(b.price * i.quantity) AS total_revenue
FROM Books b JOIN Issue i ON b.book_id = i.book_id;

-- (e) Members who joined in last 6 months
SELECT * FROM Members
WHERE membership_date >= DATE_SUB(CURDATE(), INTERVAL 6 MONTH);
```

- **Marking:** 1 mark for each correct query.

Part B: Hands-on Practical Lab Task (5 marks)

SQLite Student Management System

```
import sqlite3

try:
    conn = sqlite3.connect('school.db')
    c = conn.cursor()
```

```
    # Create table
    c.execute('''CREATE TABLE IF NOT EXISTS Students
                 (roll INTEGER PRIMARY KEY, name TEXT, class TEXT, marks INTEGER)''')

    # Insert records
    students = [(1, 'Rahul', '10A', 85), (2, 'Priya', '10A', 92),
                (3, 'Amit', '10B', 78), (4, 'Sneha', '10B', 88),
                (5, 'Vijay', '10A', 71), (6, 'Neha', '10A', 95),
                (7, 'Ravi', '10B', 82), (8, 'Pooja', '10B', 76)]
    c.executemany('INSERT OR IGNORE INTO Students VALUES (?,?,?,?)', students)

    # CRUD Operations
    # Create
    c.execute('INSERT INTO Students VALUES (9, "Kiran", "10A", 89)')

    # Read
    c.execute('SELECT * FROM Students')
    print("All Students:", c.fetchall())

    # Update
    c.execute('UPDATE Students SET marks = 90 WHERE roll = 3')

    # Delete
    c.execute('DELETE FROM Students WHERE roll = 9')

    # Aggregate Report
    c.execute('SELECT COUNT(*), AVG(marks), MAX(marks), MIN(marks) FROM Students')
    print("Stats:", c.fetchone())

    conn.commit()

except sqlite3.Error as e:
    print(f"Database error: {e}")
finally:
    conn.close()
```

- **Marking:** 1 mark for database setup, 1 mark for CRUD operations, 1 mark for aggregates, 1 mark for exception handling, 1 mark for proper closing.

Part C: Notebook Maintenance (5 marks)

1. Neatness and organisation of notebook (2 marks)
2. Completion of all assigned classwork and homework (2 marks)
3. Proper headings, dates, and indexing (1 mark)

Karnataka Residential Educational Institutions Society

ICT Class 10 - Summative Assessment 1

Summative Assessment (SA-1) - Answer Key

Class: Class 10

Assessment Type: Summative Assessment (SA-1)

Total Marks: 40

Duration: 2 hours

Topics Covered: Networking Concepts, Internet and Web Technologies, HTML Fundamentals, CSS Styling

Answer Key - Grading Guidelines

Note to Teacher: This answer key provides expected answers and marking guidelines. Accept equivalent answers where appropriate. Partial marks may be awarded for incomplete but correct responses.

Part A: MCQ (10 x 1 = 10 marks)

1. **Answer:** d - Transport layer
   - **Explanation:** The Transport layer (Layer 4) handles end-to-end communication, reliability, and flow control using TCP/UDP.
2. **Answer:** c - 192.168.1.1
   - **Explanation:** Class C addresses range from 192.0.0.0 to 223.255.255.255. 192.168.1.1 is a Class C private address.
3. **Answer:** c - Tells the browser this is an HTML5 document
   - **Explanation:** `<!DOCTYPE html>` declares the document type as HTML5, ensuring the browser renders in standards mode.
4. **Answer:** b - `padding`
   - **Explanation:** Padding creates space between the element's content and its border. Margin is outside the border.
5. **Answer:** c - `<article>`
   - **Explanation:** `<article>` represents a self-contained composition like a blog post or news article.
6. **Answer:** b - Defines character encoding
   - **Explanation:** `meta charset="UTF-8"` specifies the character encoding for the HTML document, supporting all languages.
7. **Answer:** c - 404
   - **Explanation:** HTTP 404 indicates "Not Found". 200 is OK, 301 is Moved Permanently, 500 is Internal Server Error.
8. **Answer:** c - ID selector `#main`
   - **Explanation:** ID selectors have highest specificity (1,0,0), followed by class (0,1,0), element (0,0,1), and universal (0,0,0).
9. **Answer:** a - `<video>`
   - **Explanation:** The `<video>` tag embeds video content in HTML5 with attributes like `src`, `controls`, `autoplay`.
10. **Answer:** b - Positions element relative to its normal position until scroll threshold
    - **Explanation:** `position: sticky` toggles between relative and fixed based on scroll position.

Part B: Short Answer (8 x 2 = 16 marks)

1. **IPv4 vs IPv6**
   - **Answer:** IPv4: 32-bit address, 4 billion addresses, dotted decimal notation (192.168.1.1), no built-in security. IPv6: 128-bit address, unlimited addresses, hexadecimal notation

(2001:0db8::1), built-in IPSec security. IPv6 was introduced due to IPv4 address exhaustion and need for better security.

- **Marking:** 1 mark for comparison, 1 mark for why introduced.

2. **HTML Form Code**
   - **Answer:**

```
<form action="/submit" method="POST">
  <label for="name">Full Name:</label>
  <input type="text" id="name" name="name" required><br>

  <label for="email">Email:</label>
  <input type="email" id="email" name="email" required><br>

  <label for="phone">Phone:</label>
  <input type="tel" id="phone" name="phone" pattern="[0-9]{10}" placeholder="10-digit
number"><br>

  <label for="dob">Date of Birth:</label>
  <input type="date" id="dob" name="dob"><br>

  <label>Gender:</label>
  <input type="radio" id="male" name="gender" value="male">
  <label for="male">Male</label>
  <input type="radio" id="female" name="gender" value="female">
  <label for="female">Female</label><br>

  <input type="submit" value="Submit">
</form>
```

- **Marking:** 1 mark for form structure, 1 mark for validation attributes.

3. **Block-level vs Inline elements**
   - **Answer:** Block-level: start new line, full width (div, p, h1-h6, ul, ol, li). Inline: no new line, content width only (span, a, img, strong, em). Inline-block: inline flow but accepts block styling (img, button, input).
   - **Marking:** 1 mark for each type with examples.

4. **CSS Box Model**
   - **Answer:** Content: actual content area. Padding: transparent space between content and border. Border: edge around padding. Margin: transparent space outside border. `box-sizing: border-box` includes padding and border in element's total width/height.
   - **Marking:** 0.5 marks for each component, 0.5 marks for box-sizing.

5. **TCP/IP Model**
   - **Answer:** Four layers: (1) Application - HTTP, FTP, SMTP, DNS; (2) Transport - TCP, UDP; (3) Internet - IP, ICMP, ARP; (4) Network Access - Ethernet, Wi-Fi. Each layer communicates with adjacent layers.
   - **Marking:** 1 mark for layers, 0.5 marks for each protocol.

6. **CSS Rules**
   - **Answer:**

```
/* Responsive nav */
nav { display: flex; flex-wrap: wrap; }
/* Hover underline */
a:hover { text-decoration: underline; }
```

```css
/* Media query */
@media (max-width: 768px) { nav { flex-direction: column; } }
/* 3-column grid */
.grid { display: grid; grid-template-columns: repeat(3, 1fr); }
```

- **Marking:** 0.5 marks for each rule.

7. **Responsive Web Design**
   - **Answer:** RWD creates websites that adapt to different screen sizes. Techniques: Media queries apply styles based on device width. Flexbox provides flexible layout. Relative units (%, em, vw) scale with viewport. Mobile-first approach designs for smallest screens first.
   - **Marking:** 1 mark for definition, 1 mark for techniques.
8. **Class vs ID selectors**
   - **Answer:** Class (`.name`): reusable on multiple elements, lower specificity. ID (`#name`): unique to one element, higher specificity. Specificity: ID > class > element. Example: `#header { }` overrides `.header { }` which overrides `header { }`.
   - **Marking:** 1 mark for difference, 1 mark for specificity.

Part C: Long Answer (4 x 3 = 12 marks)

1. **Student Management System HTML Page**
   - **Answer:** Complete page with: HTML5 boilerplate, semantic header with school name/logo, form with input validation (name, roll, class, section, marks), styled table with 5 rows, CSS with borders, colours, hover effects, responsive footer. Key marks: proper structure, working form, styled table, responsive design.
   - **Marking:** 1 mark for structure, 1 mark for form, 1 mark for table/CSS.
2. **Bus, Star, and Mesh Topologies**
   - **Answer:**
   - Bus: Linear backbone cable, all devices connect to it. Data travels both directions. Advantages: simple, cheap. Disadvantages: single point of failure, hard to troubleshoot. Best for small networks.
   - Star: Central hub, all devices connect to it. Data passes through hub. Advantages: easy to install, fault isolation. Disadvantages: hub failure, more cable. Best for offices.
   - Mesh: Every device connects to every other. Data takes multiple paths. Advantages: redundant, reliable. Disadvantages: expensive, complex. Best for critical systems.
   - **Marking:** 1 mark for each topology (structure, flow, pros/cons).
3. **Restaurant Menu Webpage**
   - **Answer:** Complete CSS-styled page with: hero section with gradient background, menu section using CSS Grid, food items in categories, responsive design with media queries, typography with Google Fonts, colour scheme matching restaurant theme.
   - **Marking:** 1 mark for hero, 1 mark for menu layout, 1 mark for responsiveness/styling.
4. **DNS Resolution Process**
   - **Answer:** DNS translates domain names to IP addresses. Types: Recursive (queries on behalf of client), Root (directs to TLD servers), TLD (manages .com, .org etc.), Authoritative (final answer). Process: Browser checks cache → OS cache → Recursive resolver → Root server → TLD server → Authoritative server → Returns IP to browser.
   - **Marking:** 1 mark for DNS purpose, 1 mark for server types, 1 mark for process.

Part D: Application Based (2 x 1 = 2 marks)

1. **Responsive Navigation Fix**
   - **Answer:** Problem: Nav disappears entirely on mobile. Fix: Use hamburger menu pattern.

```
nav { display: flex; flex-wrap: wrap; }
.nav-links { display: flex; list-style: none; }
@media (max-width: 768px) {
  .nav-links { display: none; }
  .menu-toggle { display: block; }
  .nav-links.active { display: flex; flex-direction: column; }
}
```

- **Marking:** 1 mark for identifying problem, 1 mark for solution.

2. **Office Network Recommendation**
   - **Answer:** Use WAN (Wide Area Network) to connect offices across cities. Protocols: TCP/IP for reliable data transfer, VPN for secure communication. Routers at each office connect to ISP. Video conferencing uses RTP/RTCP protocols.
   - **Marking:** 0.5 marks for network type, 0.5 marks for protocols.

Karnataka Residential Educational Institutions Society

ICT Class 10 - Summative Assessment 2

Summative Assessment (SA-2) - Answer Key

Class: Class 10

Assessment Type: Summative Assessment (SA-2)

Total Marks: 40

Duration: 2 hours

Topics Covered: Ch 1-6: Networking, HTML/CSS, Python Programming, SQL Database Management

Answer Key - Grading Guidelines

Note to Teacher: This answer key provides expected answers and marking guidelines. Accept equivalent answers where appropriate. Partial marks may be awarded for incomplete but correct responses.

Part A: MCQ (10 x 1 = 10 marks)

1. **Answer:** b - `[1, 2, 3, 4]`
   - **Explanation:** Lists are mutable. `y = x` creates a reference, not a copy. Modifying `y` also modifies `x`.
2. **Answer:** b - `ALTER TABLE ... ADD`
   - **Explanation:** `ALTER TABLE table_name ADD column_name datatype` adds a new column to an existing table.
3. **Answer:** a - `font-weight: bold`
   - **Explanation:** `font-weight: bold` makes text bold. There is no `text-style: bold` or `text-weight: bold` in CSS.
4. **Answer:** b - Handles exceptions gracefully
   - **Explanation:** `try-except` catches and handles exceptions, preventing program crashes.
5. **Answer:** b - `<a>`
   - **Explanation:** The `<a>` tag creates hyperlinks with `href` attribute specifying the URL.
6. **Answer:** d - Both B and C
   - **Explanation:** Both `NOW()` and `CURRENT_TIMESTAMP` return the current date and time.
7. **Answer:** b - `World`
   - **Explanation:** `"Hello World".split()` splits by whitespace into `["Hello", "World"]`. Index 1 is "World".
8. **Answer:** b - `visibility: hidden`
   - **Explanation:** `visibility: hidden` hides the element but preserves its space. `display: none` removes it from flow.
9. **Answer:** a - `raise`
   - **Explanation:** Python uses `raise` keyword to re-throw caught exceptions. `throw` is Java/C++.
10. **Answer:** b - `DESCRIBE table_name`
    - **Explanation:** `DESCRIBE` (or `DESC`) shows the structure of a table including columns, types, and constraints.

Part B: Short Answer (8 x 2 = 16 marks)

1. **CSV Reading Program**
   - **Answer:**

```
import csv

with open('students.csv', 'r') as f:
    reader = csv.DictReader(f)
    students = list(reader)
```

```
# Sort by marks descending
students.sort(key=lambda x: int(x['Marks']), reverse=True)

# Display top 3
print("Top 3 Scorers:")
for i, s in enumerate(students[:3], 1):
    print(f"{i}. {s['Name']} - Roll: {s['Roll']} - Marks: {s['Marks']}")
```

- **Marking:** 1 mark for CSV reading, 1 mark for sorting and display.

2. **JSON Output**
   - **Answer:**

```
{
  "students": [
    {
      "name": "A",
      "marks": 85
    },
    {
      "name": "B",
      "marks": 92
    }
  ]
}
B
```

`json.dumps()` converts dict to formatted JSON string. `json.loads()` parses JSON back to dict. Indexing accesses nested values.

- **Marking:** 1 mark for output, 1 mark for explanation.

3. **Online Shopping Queries**
   - **Answer:**

```
-- (a) Customers from Bangalore
SELECT * FROM Customers WHERE city = 'Bangalore';

-- (b) Products with stock less than 10
SELECT * FROM Products WHERE stock < 10;

-- (c) Total orders by each customer
SELECT cid, COUNT(*) AS total_orders FROM Orders GROUP BY cid;

-- (d) Most ordered product
SELECT pid, SUM(quantity) AS total_qty FROM Orders GROUP BY pid ORDER BY total_qty
DESC LIMIT 1;
```

- **Marking:** 0.5 marks for each query.

4. **List vs Tuple**
   - **Answer:** List: mutable, [], square brackets, slower, dynamic. Tuple: immutable, (), parentheses, faster, fixed. Use lists when data changes; use tuples for fixed data or dictionary keys.
   - **Marking:** 1 mark for differences, 1 mark for use cases.

5. **Pricing Card CSS**
   - **Answer:**

```
.card { box-shadow: 0 4px 8px rgba(0,0,0,0.1); border-radius: 10px; }
.card-header { background: linear-gradient(135deg, #667eea, #764ba2); color: white; }
```

```
.price { font-size: 2.5em; font-weight: bold; }
.features li { list-style: none; }
.features li::before { content: "✓ "; color: green; }
.cta-btn { background: #667eea; color: white; border: none; padding: 10px 20px; }
.cta-btn:hover { background: #764ba2; }
```

- **Marking:** 1 mark for card structure, 1 mark for styling.

6. **Normalization**
   - **Answer:** Normalization organizes data to reduce redundancy. 1NF: atomic values, no repeating groups. 2NF: 1NF + no partial dependencies. 3NF: 2NF + no transitive dependencies. Example violation: 1NF - phone numbers in comma-separated string; 2NF - student name depends only on roll, not subject; 3NF - department name depends on department_id, not roll.
   - **Marking:** 1 mark for definition, 1 mark for NF differences.

7. **Merge Sort**
   - **Answer:**

```
def merge_sort(arr):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])

    return merge(left, right)

def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result
```

- **Marking:** 1 mark for split logic, 1 mark for merge logic.

8. **SQL JOINs**
   - **Answer:** JOIN combines rows from multiple tables. INNER JOIN: only matching rows. LEFT JOIN: all rows from left table, matching from right. RIGHT JOIN: all rows from right table, matching from left.

```
SELECT Students.name, Marks.marks
FROM Students
LEFT JOIN Marks ON Students.roll = Marks.roll;
```

- **Marking:** 1 mark for JOIN types, 1 mark for example.

Part C: Long Answer (4 x 3 = 12 marks)

1. **Library Management System**

- **Answer:** Complete Python program with: file handling for book records, functions for add_book, search_book, issue_book, return_book, display_all, try-except for file operations, daily report generation. Key elements: proper file modes, dictionary storage, input validation.
- **Marking:** 1 mark for file handling, 1 mark for functions, 1 mark for exception handling.

2. **Healthcare Appointment System**
   - **Answer:** Complete HTML/CSS page with: header with hospital name and nav, appointment form with doctor selection and validation, styled table showing doctors with specializations, responsive media queries, professional medical colour scheme (blues/greens).
   - **Marking:** 1 mark for structure, 1 mark for form/table, 1 mark for styling.

3. **School Examination System Queries**
   - **Answer:**

```sql
-- (a) Result card
SELECT s.name, sub.sub_name, m.marks_obtained,
       SUM(m.marks_obtained) OVER(PARTITION BY s.roll) AS total,
       (SUM(m.marks_obtained) OVER(PARTITION BY s.roll) / (SELECT SUM(max_marks) FROM
Subjects)) * 100 AS percentage
FROM Students s JOIN Marks m ON s.roll = m.roll JOIN Subjects sub ON m.sub_id =
sub.sub_id;

-- (b) Class topper
SELECT name FROM Students WHERE roll = (SELECT roll FROM Marks GROUP BY roll ORDER BY
SUM(marks_obtained) DESC LIMIT 1);

-- (c) Subject-wise toppers
SELECT sub_name, name FROM Subjects sub JOIN Marks m ON sub.sub_id = m.sub_id JOIN
Students s ON m.roll = s.roll
WHERE m.marks_obtained = (SELECT MAX(marks_obtained) FROM Marks WHERE sub_id =
sub.sub_id);

-- (d) Failed students (marks < 35% of max)
SELECT DISTINCT name FROM Students WHERE roll IN (SELECT roll FROM Marks m JOIN
Subjects sub ON m.sub_id = sub.sub_id WHERE m.marks_obtained < sub.max_marks * 0.35);

-- (e) Overall pass percentage
SELECT (COUNT(DISTINCT CASE WHEN marks_obtained >= 35 THEN roll END) / COUNT(DISTINCT
roll)) * 100 FROM Marks;

-- (f) Rank list
SELECT RANK() OVER(ORDER BY SUM(marks_obtained) DESC) AS rank, name,
SUM(marks_obtained) AS total
FROM Students JOIN Marks ON Students.roll = Marks.roll GROUP BY roll ORDER BY total
DESC;
```

- **Marking:** 0.5 marks for each query.

4. **Data Communication Process**
   - **Answer:** Components: Sender (source device), Receiver (destination), Medium (cable/wireless), Protocol (rules). Transmission types: Simplex (one-way), Duplex (two-way), Multiplex (multiple signals). Error detection: Parity check (counts bits), CRC (polynomial division). TCP role: segments data, adds sequence numbers, retransmits lost packets, uses checksums for integrity.
   - **Marking:** 1 mark for components, 1 mark for transmission types, 1 mark for error detection and TCP.

Part D: Application Based (2 x 1 = 2 marks)

1. **File Handling Fix**
   - **Answer:** Issue: No exception handling for missing file, manual close. Fix:

```
try:
    with open("data.txt") as f:
        content = f.read()
        print(content)
except FileNotFoundError:
    print("File not found!")
```

`with` is preferred because it automatically closes the file even if exceptions occur.

- **Marking:** 1 mark for try-except, 1 mark for with statement explanation.

2. **Section-wise Query**
   - **Answer:**

```
SELECT section, AVG(marks) AS avg_marks,
       (SELECT name FROM Students s2 WHERE s2.section = s1.section ORDER BY marks
DESC LIMIT 1) AS topper
FROM Students s1
GROUP BY section;
```

- **Marking:** 1 mark for correct query, 1 mark for clause explanation.

Karnataka Residential Educational Institutions Society

ICT Class 10 - Unit Test 1

Unit Test 1 - Answer Key

Class: Class 10

Assessment Type: Unit Test 1

Total Marks: 10

Duration: 30 minutes

Topics Covered: Networking Concepts

Answer Key - Grading Guidelines

Note to Teacher: This answer key provides expected answers and marking guidelines. Accept equivalent answers where appropriate. Partial marks may be awarded for incomplete but correct responses.

Section I: MCQ (5 x 1 = 5 marks)

1. **Answer:** b - Star topology
   - **Explanation:** Star topology uses a central hub or switch to connect all devices. All data passes through the central node.
2. **Answer:** c - 7
   - **Explanation:** The OSI (Open Systems Interconnection) model has 7 layers: Physical, Data Link, Network, Transport, Session, Presentation, Application.
3. **Answer:** b - 172.16.0.1
   - **Explanation:** Class B IP addresses range from 128.0.0.0 to 191.255.255.255. 172.16.0.1 falls in this range. 192.168.x.x is Class C, 10.x.x.x is Class A.
4. **Answer:** c - SMTP
   - **Explanation:** SMTP (Simple Mail Transfer Protocol) is used to send emails. POP3 and IMAP are used to receive emails. HTTP is for web browsing, FTP for file transfer.
5. **Answer:** b - To filter incoming and outgoing network traffic
   - **Explanation:** A firewall monitors and controls network traffic based on security rules, blocking unauthorized access while allowing legitimate communication.

Section II: Short Answer (5 x 1 = 5 marks)

1. **IPv4 vs IPv6**
   - **Answer:** IPv4 uses 32-bit addresses (e.g., 192.168.1.1), providing 4 billion addresses. IPv6 uses 128-bit addresses (e.g., 2001:0db8:85a3::8a2e:0370:7334), providing virtually unlimited addresses. IPv6 was introduced because IPv4 addresses are exhausted. IPv6 also has better security and routing.
   - **Marking:** 1 mark for comparison, 1 mark for why needed.
2. **Star Topology diagram**
   - **Answer:** Diagram should show a central hub/switch with devices connected to it like spokes. Advantages: Easy to install, failure of one device doesn't affect others. Disadvantages: If hub fails, entire network goes down; requires more cable than bus topology.
   - **Marking:** 1 mark for diagram, 0.5 marks for each advantage/disadvantage.
3. **Full forms and purposes**
   - **Answer:** HTTP (HyperText Transfer Protocol) - transfers web pages. FTP (File Transfer Protocol) - transfers files between computers. SMTP (Simple Mail Transfer Protocol) - sends emails. DNS (Domain Name System) - translates domain names to IP addresses. TCP (Transmission Control Protocol) - ensures reliable data delivery.
   - **Marking:** 0.2 marks for each full form and purpose.
4. **MAC address vs IP address**

- **Answer:** MAC address is a hardware address burned into the network card (e.g., 00:1A:2B:3C:4D:5E). IP address is a logical address assigned by the network (e.g., 192.168.1.1). MAC is permanent, IP changes with network. MAC works at Data Link layer, IP works at Network layer.
- **Marking:** 1 mark for each comparison point.

5. **Data travel through Internet**
   - **Answer:** When you send data: (1) Your computer creates packets with source/destination IP. (2) Packets travel through NIC to router. (3) Router examines IP and forwards to next hop using routing table. (4) Packets traverse multiple routers and ISPs. (5) At destination, packets are reassembled. Protocols: TCP ensures reliable delivery, IP handles addressing, HTTP/FTP for application data. Devices: NIC, router, switch, modem.
   - **Marking:** 1 mark for process, with at least 3 devices/protocols mentioned.

Karnataka Residential Educational Institutions Society

ICT Class 10 - Unit Test 2

Unit Test 2 - Answer Key

Class: Class 10

Assessment Type: Unit Test 2

Total Marks: 10

Duration: 30 minutes

Topics Covered: HTML and CSS

Answer Key - Grading Guidelines

Note to Teacher: This answer key provides expected answers and marking guidelines. Accept equivalent answers where appropriate. Partial marks may be awarded for incomplete but correct responses.

Section I: MCQ (5 x 1 = 5 marks)

1. **Answer:** b - `<style>`
   - **Explanation:** The `<style>` tag defines internal CSS within the HTML document. `<link>` links external stylesheets, `<script>` is for JavaScript.
2. **Answer:** a - `<nav>`
   - **Explanation:** HTML5 introduced the `<nav>` semantic element specifically for navigation sections.
3. **Answer:** b - `block`
   - **Explanation:** `<div>` is a block-level element by default, meaning it takes full width and starts on a new line.
4. **Answer:** b - `em`
   - **Explanation:** `em` is relative to the element's font-size. `px` is absolute, `cm` and `pt` are physical units.
5. **Answer:** c - `:hover`
   - **Explanation:** The `:hover` pseudo-class applies styles when the user's mouse pointer is over an element.

Section II: Short Answer (5 x 1 = 5 marks)

1. **HTML Form Code**
   - **Answer:**

```
<form>
  <label for="username">Username:</label>
  <input type="text" id="username" name="username" placeholder="Enter username"><br>

  <label for="password">Password:</label>
  <input type="password" id="password" name="password" placeholder="Enter
password"><br>

  <label for="email">Email:</label>
  <input type="email" id="email" name="email" placeholder="Enter email"><br>

  <input type="submit" value="Submit">
</form>
```

- **Marking:** 1 mark for form structure, 1 mark for labels and placeholders.

2. **Relative vs Absolute positioning**
   - **Answer:** Relative positioning moves element from its normal position but space is reserved. Absolute positioning removes element from flow and positions relative to nearest positioned ancestor. Use relative for small adjustments; use absolute for overlay elements like dropdown menus or modals.

- **Marking:** 1 mark for difference, 1 mark for use cases.

3. **z-index property**
   - **Answer:** `z-index` controls the stacking order of positioned elements (higher values appear in front). It only works on elements with `position` set to relative, absolute, fixed, or sticky. It does not work on statically positioned elements.
   - **Marking:** 1 mark for purpose, 1 mark for condition.

4. **Responsive Grid CSS**
   - **Answer:**

```css
.grid-container {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 20px;
}

@media (max-width: 768px) {
  .grid-container {
    grid-template-columns: 1fr;
  }
}
```

- **Marking:** 1 mark for grid setup, 1 mark for media query.

5. **margin: auto vs text-align: center**
   - **Answer:** `margin: auto` horizontally centers block-level elements within their container by distributing equal margin on both sides. `text-align: center` centers inline content (text, images) within a block element. Use `margin: auto` for divs; use `text-align: center` for text.
   - **Marking:** 1 mark for each explanation and use case.

Karnataka Residential Educational Institutions Society

ICT Class 10 - Unit Test 3

Unit Test 3 - Answer Key

Class: Class 10

Assessment Type: Unit Test 3

Total Marks: 10

Duration: 30 minutes

Topics Covered: Python Programming

Answer Key - Grading Guidelines

Note to Teacher: This answer key provides expected answers and marking guidelines. Accept equivalent answers where appropriate. Partial marks may be awarded for incomplete but correct responses.

Section I: MCQ (5 x 1 = 5 marks)

1. **Answer:** c - `YTH`
   - **Explanation:** `s[1:4]` extracts characters from index 1 to 3 (excluding 4). "PYTHON" → index 1='Y', 2='T', 3='H' → "YTH".
2. **Answer:** a - `{}`
   - **Explanation:** `{}` creates an empty dictionary. `[]` creates a list, `()` creates a tuple.
3. **Answer:** b - Removes leading and trailing whitespace
   - **Explanation:** `strip()` removes whitespace from both ends of a string. It does not remove spaces in the middle.
4. **Answer:** b - `re`
   - **Explanation:** Python's built-in module for regular expressions is `re`. It provides functions like `match()`, `search()`, `findall()`.
5. **Answer:** a - `[1, 2, 3, 1, 2, 3]`
   - **Explanation:** `[1, 2, 3] * 2` repeats the list twice, resulting in `[1, 2, 3, 1, 2, 3]`.

Section II: Short Answer (5 x 1 = 5 marks)

1. **Output and explanation**
   - **Answer:**

```
1. apple
2. banana
3. cherry
```

`enumerate()` returns index-item pairs. `i` gets the index (0,1,2), `fruit` gets the value. `i+1` starts numbering from 1. f-string formats the output.

- **Marking:** 1 mark for output, 1 mark for explanation.

2. **split() vs join()**
   - **Answer:** `split()` divides a string into a list based on a delimiter. Example: `"hello world".split()` → `["hello", "world"]`. `join()` combines list elements into a string with a separator. Example: `" ".join(["hello", "world"])` → `"hello world"`.
   - **Marking:** 1 mark for each method with example.
3. **Palindrome function**
   - **Answer:**

```python
def is_palindrome(s):
    s = s.lower()
    return s == s[::-1]

# Test
print(is_palindrome("Madam"))  # True
```

```
print(is_palindrome("hello"))  # False
print(is_palindrome("Racecar"))  # True
```

- **Marking:** 1 mark for function, 0.5 marks for each test case.

4. **List comprehension vs Generator expression**
   - **Answer:** List comprehension creates a complete list in memory: `[x**2 for x in range(10)]`. Generator expression yields items lazily: `(x**2 for x in range(10))`. Generators are memory-efficient for large data. List comprehensions are faster for small datasets.
   - **Marking:** 1 mark for difference, 1 mark for examples.

5. **Dictionary output**
   - **Answer:**

```
['a', 'b', 'c']
[1, 2, 3]
0
```

`d.keys()` returns all keys, `d.values()` returns all values, `d.get("d", 0)` returns 0 (default) since "d" is not in the dictionary.

- **Marking:** 0.5 marks for each correct output.

Karnataka Residential Educational Institutions Society

ICT Class 10 - Unit Test 4

Unit Test 4 - Answer Key

Class: Class 10

Assessment Type: Unit Test 4

Total Marks: 10

Duration: 30 minutes

Topics Covered: SQL Database Management

Answer Key - Grading Guidelines

Note to Teacher: This answer key provides expected answers and marking guidelines. Accept equivalent answers where appropriate. Partial marks may be awarded for incomplete but correct responses.

Section I: MCQ (5 x 1 = 5 marks)

1. **Answer:** b - `ORDER BY`
   - **Explanation:** `ORDER BY` sorts the result set in ascending or descending order. There is no `SORT BY` or `ARRANGE BY` in SQL.
2. **Answer:** c - `CREATE`
   - **Explanation:** `CREATE` is a DDL (Data Definition Language) command used to create database objects. `SELECT`, `INSERT`, `UPDATE` are DML commands.
3. **Answer:** a - `SELECT - FROM - WHERE - GROUP BY - ORDER BY`
   - **Explanation:** The correct SQL clause order is: SELECT, FROM, WHERE, GROUP BY, HAVING, ORDER BY.
4. **Answer:** b - `COUNT()`
   - **Explanation:** `COUNT()` returns the number of rows. `SUM()` returns total, there is no `TOTAL()` or `ROWS()` function.
5. **Answer:** b - `UNIQUE`
   - **Explanation:** `UNIQUE` constraint ensures all values in a column are different. `PRIMARY KEY` also ensures uniqueness but cannot contain NULLs.

Section II: Short Answer (5 x 1 = 5 marks)

1. **SQL Queries for Products table**
   - **Answer:**

```sql
-- Products with price between 100 and 500
SELECT * FROM Products WHERE price BETWEEN 100 AND 500;

-- Count products in each category
SELECT category, COUNT(*) FROM Products GROUP BY category;

-- Product with maximum price
SELECT * FROM Products WHERE price = (SELECT MAX(price) FROM Products);

-- Products sorted by stock descending
SELECT * FROM Products ORDER BY stock DESC;
```

- **Marking:** 0.5 marks for each correct query.

2. **Foreign key**
   - **Answer:** A foreign key is a column that references the primary key of another table. It establishes a parent-child relationship. Example: `Orders.customer_id` references `Customers.customer_id`. This ensures referential integrity - you cannot add an order for a non-existent customer.
   - **Marking:** 1 mark for definition, 1 mark for example and relationship.

3. **SQL Output**
   - **Answer:**

```
SELECT ROUND(45.678, 2);     -- 45.68
SELECT MOD(17, 5);           -- 2
SELECT NOW();                -- Current date and time (e.g., 2024-01-15 10:30:00)
```

- **Marking:** 0.5 marks for each correct output.

4. **DELETE vs DROP**
   - **Answer:** `DELETE` removes rows from a table (DML command, can be rolled back with WHERE clause). `DROP` removes the entire table including structure (DDL command, cannot be rolled back). DELETE: `DELETE FROM Students WHERE roll=1`. DROP: `DROP TABLE Students`.
   - **Marking:** 1 mark for each difference.

5. **JOIN with example**
   - **Answer:** JOIN combines rows from two tables based on a related column. INNER JOIN returns only matching rows.

```
SELECT Students.name, Marks.marks
FROM Students
INNER JOIN Marks ON Students.roll = Marks.roll;
```

- **Marking:** 1 mark for JOIN explanation, 1 mark for correct query.

Karnataka Residential Educational Institutions Society

ICT Class 10 - Practical Lab Exam 1

Practical Lab Exam 1 - Answer Key

Class: Class 10

Assessment Type: Practical Lab Exam 1

Total Marks: 20

Duration: 45 minutes

Topics Covered: HTML/CSS and Python Programming

Answer Key - Grading Guidelines

Note to Teacher: This answer key provides expected answers and marking guidelines. Accept equivalent answers where appropriate. Partial marks may be awarded for incomplete but correct responses.

Lab Tasks (15 marks)

Task 1: HTML/CSS - Personal Portfolio Website (5 marks)

Expected Solution:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Personal Portfolio</title>
  <style>
    * { margin: 0; padding: 0; box-sizing: border-box; }
    body { font-family: Arial, sans-serif; }
    .hero {
      background: linear-gradient(135deg, #667eea, #764ba2);
      color: white; text-align: center; padding: 100px 20px;
    }
    .hero h1 { font-size: 2.5em; }
    .skills { display: flex; flex-wrap: wrap; justify-content: center; padding:
40px; }
    .skill { width: 200px; margin: 10px; }
    .progress-bar { background: #ddd; border-radius: 5px; }
    .progress { background: #667eea; height: 20px; border-radius: 5px; }
    .projects { display: grid; grid-template-columns: repeat(auto-fit, minmax(250px,
1fr)); gap: 20px; padding: 40px; }
    .project-card { border: 1px solid #ddd; border-radius: 8px; padding: 15px; }
    form { max-width: 500px; margin: 40px auto; padding: 20px; }
    input, textarea { width: 100%; padding: 10px; margin: 5px 0; }
    @media (max-width: 768px) {
      .hero h1 { font-size: 1.8em; }
      .projects { grid-template-columns: 1fr; }
    }
  </style>
</head>
<body>
  <section class="hero">
    <h1>John Doe</h1>
    <p>Web Developer | Designer</p>
  </section>

  <section class="skills">
    <div class="skill"><h3>HTML</h3><div class="progress-bar"><div class="progress"
style="width:90%"></div></div></div>
    <div class="skill"><h3>CSS</h3><div class="progress-bar"><div class="progress"
style="width:85%"></div></div></div>
    <div class="skill"><h3>JavaScript</h3><div class="progress-bar"><div
```

```
class="progress" style="width:75%"></div></div></div>
    <div class="skill"><h3>Python</h3><div class="progress-bar"><div class="progress"
style="width:80%"></div></div></div>
    <div class="skill"><h3>SQL</h3><div class="progress-bar"><div class="progress"
style="width:70%"></div></div></div>
  </section>

  <section class="projects">
    <div class="project-card"><h3>Project 1</h3><p>Description</p></div>
    <div class="project-card"><h3>Project 2</h3><p>Description</p></div>
    <div class="project-card"><h3>Project 3</h3><p>Description</p></div>
  </section>

  <form>
    <input type="text" placeholder="Name">
    <input type="email" placeholder="Email">
    <textarea placeholder="Message"></textarea>
    <button type="submit">Send</button>
  </form>
</body>
</html>
```

Marking Criteria:

- Hero section with name and tagline: 1 mark
- Skills with progress bars: 1 mark
- Project cards (minimum 3): 1 mark
- Contact form: 1 mark
- Responsive design with media queries: 1 mark

Task 2: Python - Student Records Manager (5 marks)

Expected Solution:

```python
students = [
    {"name": "Rahul", "roll": 1, "marks": [85, 90, 78]},
    {"name": "Priya", "roll": 2, "marks": [92, 88, 95]},
    {"name": "Amit", "roll": 3, "marks": [78, 82, 75]}
]

def add_student():
    name = input("Enter name: ")
    roll = int(input("Enter roll number: "))
    marks = [int(input(f"Subject {i+1}: ")) for i in range(3)]
    students.append({"name": name, "roll": roll, "marks": marks})
    print("Student added!")

def display_all():
    print(f"{'Name':<15}{'Roll':<8}{'Total':<8}{'Avg':<8}")
    for s in students:
        total = sum(s["marks"])
        avg = total / 3
        print(f"{s['name']:<15}{s['roll']:<8}{total:<8}{avg:<8.2f}")

def search_student():
    roll = int(input("Enter roll number: "))
    for s in students:
        if s["roll"] == roll:
```

```
            print(f"Found: {s['name']}, Marks: {s['marks']}")
            return
    print("Student not found!")

def sorted_by_total():
    sorted_students = sorted(students, key=lambda x: sum(x["marks"]), reverse=True)
    display_all()

def statistics():
    totals = [sum(s["marks"]) for s in students]
    print(f"Average: {sum(totals)/len(totals):.2f}")
    print(f"Highest: {max(totals)}")
    print(f"Lowest: {min(totals)}")

while True:
    print("\n1.Add 2.Display 3.Search 4.Sorted 5.Stats 6.Exit")
    choice = input("Enter choice: ")
    if choice == "1": add_student()
    elif choice == "2": display_all()
    elif choice == "3": search_student()
    elif choice == "4": sorted_by_total()
    elif choice == "5": statistics()
    elif choice == "6": break
```

Marking Criteria:

- List of dictionaries structure: 1 mark
- Menu-driven interface: 1 mark
- Search and sort functions: 1 mark
- Statistics calculation: 1 mark
- Input validation and formatting: 1 mark

Task 3: Python - File Handling with Exception (5 marks)

Expected Solution:

```
import collections

def analyze_file(filename):
    try:
        with open(filename, 'r') as f:
            content = f.read()

        lines = content.split('\n')
        words = content.split()
        chars = len(content)
        unique_words = set(words)

        word_count = collections.Counter(words)
        top5 = word_count.most_common(5)

        with open('summary.txt', 'w') as f:
            f.write(f"File Analysis Report\n")
            f.write(f"{'='*30}\n")
            f.write(f"Total Lines: {len(lines)}\n")
            f.write(f"Total Words: {len(words)}\n")
            f.write(f"Total Characters: {chars}\n")
            f.write(f"Unique Words: {len(unique_words)}\n")
```

```
            f.write(f"\nTop 5 Words:\n")
            for word, count in top5:
                f.write(f"  {word}: {count}\n")

        print("Summary written to summary.txt")

    except FileNotFoundError:
        print(f"Error: File '{filename}' not found!")
    except PermissionError:
        print(f"Error: No permission to read '{filename}'!")
    except Exception as e:
        print(f"Error: {str(e)}")

filename = input("Enter filename: ")
analyze_file(filename)
```

Marking Criteria:

- File reading with with statement: 1 mark
- Counting logic (words, lines, characters): 1 mark
- Unique words and frequency: 1 mark
- Exception handling (3 types): 1 mark
- Report writing to file: 1 mark

Viva Voce (5 marks)

1. **display: none vs visibility: hidden**
   - **Answer:** `display: none` removes element from document flow (no space reserved). `visibility: hidden` hides element but space is preserved. Use `display: none` when element should not affect layout; use `visibility: hidden` when you want to toggle visibility without layout shift.
   - **Marking:** 1 mark for correct explanation.
2. **CSS Box Model**
   - **Answer:** Box model: content → padding → border → margin. `box-sizing: border-box` includes padding and border in the element's total width/height, making layout calculations easier.
   - **Marking:** 1 mark for box model explanation.
3. **with open() preference**
   - **Answer:** `with open()` automatically closes the file after the block, even if exceptions occur. Manual open()/close() requires explicit close and may leak resources if exceptions happen before close().
   - **Marking:** 1 mark for explanation.
4. **read(), readline(), readlines()**
   - **Answer:** `read()` returns entire file as string. `readline()` returns one line. `readlines()` returns list of all lines. Use `read()` for small files, `readline()` for line-by-line, `readlines()` when list needed.
   - **Marking:** 1 mark for differences.
5. **Python exception handling**
   - **Answer:** Python uses `try-except-else-finally`. `try` contains risky code, `except` handles errors, `else` runs if no exception, `finally` always runs (cleanup). Purpose: gracefully handle errors without crashing.

- **Marking:** 1 mark for explanation.

Karnataka Residential Educational Institutions Society

ICT Class 10 - Practical Lab Exam 2

Practical Lab Exam 2 - Answer Key

Class: Class 10

Assessment Type: Practical Lab Exam 2

Total Marks: 20

Duration: 45 minutes

Topics Covered: SQL Database and Python with Database

Answer Key - Grading Guidelines

Note to Teacher: This answer key provides expected answers and marking guidelines. Accept equivalent answers where appropriate. Partial marks may be awarded for incomplete but correct responses.

Lab Tasks (15 marks)

Task 1: SQL Queries (5 marks)

Expected Solution:

```sql
-- (a) Books with price above 500 sorted by title
SELECT * FROM Books WHERE price > 500 ORDER BY title ASC;

-- (b) Customers from Bangalore or Mysore
SELECT * FROM Customers WHERE city IN ('Bangalore', 'Mysore');

-- (c) Count books in each genre
SELECT genre, COUNT(*) AS book_count FROM Books GROUP BY genre;

-- (d) Total revenue for each book
SELECT b.title, SUM(b.price * s.quantity) AS total_revenue
FROM Books b JOIN Sales s ON b.book_id = s.book_id
GROUP BY b.book_id, b.title;

-- (e) Top 3 selling books by quantity
SELECT b.title, SUM(s.quantity) AS total_sold
FROM Books b JOIN Sales s ON b.book_id = s.book_id
GROUP BY b.book_id, b.title
ORDER BY total_sold DESC
LIMIT 3;

-- (f) Customers who haven't purchased
SELECT * FROM Customers
WHERE cust_id NOT IN (SELECT DISTINCT cust_id FROM Sales);

-- (g) Monthly sales summary
SELECT MONTH(sale_date) AS month, SUM(price * quantity) AS total_sales, SUM(quantity)
AS total_books
FROM Sales JOIN Books ON Sales.book_id = Books.book_id
GROUP BY MONTH(sale_date);
```

Marking Criteria:

- Each query: 0.7 marks (7 queries × 0.7 = 5 marks)

Task 2: Python with SQLite Database (5 marks)

Expected Solution:

```python
import sqlite3

try:
    conn = sqlite3.connect('result.db')
    c = conn.cursor()
```

```
    # Create tables
    c.execute('''CREATE TABLE IF NOT EXISTS Students
                 (roll INTEGER PRIMARY KEY, name TEXT, class TEXT)''')
    c.execute('''CREATE TABLE IF NOT EXISTS Subjects
                 (sub_id INTEGER PRIMARY KEY, sub_name TEXT, max_marks INTEGER)''')
    c.execute('''CREATE TABLE IF NOT EXISTS Marks
                 (roll INTEGER, sub_id INTEGER, marks INTEGER,
                  FOREIGN KEY(roll) REFERENCES Students(roll),
                  FOREIGN KEY(sub_id) REFERENCES Subjects(sub_id))''')

    # Insert subjects
    subjects = [(1, 'Math', 100), (2, 'Science', 100), (3, 'English', 100), (4,
'Kannada', 100)]
    c.executemany('INSERT OR IGNORE INTO Subjects VALUES (?,?,?)', subjects)

    # Insert students and marks
    students = [(1, 'Rahul', '10A'), (2, 'Priya', '10A'), (3, 'Amit', '10B'),
                (4, 'Sneha', '10B'), (5, 'Vijay', '10A'), (6, 'Neha', '10A'),
                (7, 'Ravi', '10B'), (8, 'Pooja', '10B')]
    c.executemany('INSERT OR IGNORE INTO Students VALUES (?,?,?)', students)

    marks_data = [
        (1, 1, 85), (1, 2, 90), (1, 3, 78), (1, 4, 82),
        (2, 1, 92), (2, 2, 88), (2, 3, 95), (2, 4, 90),
        (3, 1, 78), (3, 2, 72), (3, 3, 80), (3, 4, 75),
        (4, 1, 88), (4, 2, 85), (4, 3, 82), (4, 4, 88),
        (5, 1, 71), (5, 2, 68), (5, 3, 75), (5, 4, 70),
        (6, 1, 95), (6, 2, 92), (6, 3, 98), (6, 4, 95),
        (7, 1, 82), (7, 2, 78), (7, 3, 85), (7, 4, 80),
        (8, 1, 76), (8, 2, 72), (8, 3, 78), (8, 4, 75)
    ]
    c.executemany('INSERT INTO Marks VALUES (?,?,?)', marks_data)

    # 1. Display all students with marks
    c.execute('''SELECT s.name, sub.sub_name, m.marks
                 FROM Students s JOIN Marks m ON s.roll = m.roll
                 JOIN Subjects sub ON m.sub_id = sub.sub_id
                 ORDER BY s.roll, sub.sub_id''')
    print("Student Marks:")
    for row in c.fetchall():
        print(f"  {row[0]} - {row[1]}: {row[2]}")

    # 2. Total and percentage
    c.execute('''SELECT s.name, SUM(m.marks) AS total,
                 (SUM(m.marks) / 400.0) * 100 AS percentage
                 FROM Students s JOIN Marks m ON s.roll = m.roll
                 GROUP BY s.roll ORDER BY total DESC''')
    print("\nTotals:")
    for row in c.fetchall():
        print(f"  {row[0]}: Total={row[1]}, Percentage={row[2]:.1f}%")

    # 3. Class topper
    c.execute('''SELECT s.name, SUM(m.marks) AS total
                 FROM Students s JOIN Marks m ON s.roll = m.roll
                 GROUP BY s.roll ORDER BY total DESC LIMIT 1''')
```

```
    topper = c.fetchone()
    print(f"\nTopper: {topper[0]} with {topper[1]} marks")

    # 4. Subject-wise toppers
    c.execute('''SELECT sub.sub_name, s.name, m.marks
                 FROM Subjects sub JOIN Marks m ON sub.sub_id = m.sub_id
                 JOIN Students s ON m.roll = s.roll
                 WHERE m.marks = (SELECT MAX(marks) FROM Marks WHERE sub_id =
sub.sub_id)''')
    print("\nSubject Toppers:")
    for row in c.fetchall():
        print(f"  {row[0]}: {row[1]} ({row[2]})")

    # 5. Failed students (percentage < 40%)
    c.execute('''SELECT s.name, (SUM(m.marks)/400.0)*100 AS pct
                 FROM Students s JOIN Marks m ON s.roll = m.roll
                 GROUP BY s.roll HAVING pct < 40''')
    print("\nFailed Students:")
    for row in c.fetchall():
        print(f"  {row[0]}: {row[1]:.1f}%")

    # 6. Rank list
    c.execute('''SELECT RANK() OVER(ORDER BY SUM(m.marks) DESC) AS rank,
                 s.name, SUM(m.marks) AS total
                 FROM Students s JOIN Marks m ON s.roll = m.roll
                 GROUP BY s.roll''')
    print("\nRank List:")
    for row in c.fetchall():
        print(f"  Rank {row[0]}: {row[1]} ({row[2]})")

    conn.commit()

except sqlite3.Error as e:
    print(f"Database error: {e}")
finally:
    if conn:
        conn.close()
```

Marking Criteria:

- Database and table creation: 1 mark
- Data insertion: 1 mark
- Query operations (display, calculate, topper): 1 mark
- Subject-wise toppers and failed students: 1 mark
- Rank list and exception handling: 1 mark

Task 3: Python - Data Analysis with CSV (5 marks)

Expected Solution:

```
import csv
import collections

def analyze_sales():
    try:
        with open('sales_data.csv', 'r') as f:
            reader = csv.DictReader(f)
            data = list(reader)
```

```
        # 1. Total sales by region
        region_sales = collections.defaultdict(float)
        for row in data:
            region_sales[row['Region']] += int(row['Quantity']) * float(row['Price'])

        # 2. Best-selling product
        product_qty = collections.defaultdict(int)
        for row in data:
            product_qty[row['Product']] += int(row['Quantity'])
        best_product = max(product_qty, key=product_qty.get)

        # 3. Daily sales trend
        daily_sales = collections.defaultdict(float)
        for row in data:
            daily_sales[row['Date']] += int(row['Quantity']) * float(row['Price'])

        # 4. Region with highest average order value
        region_orders = collections.defaultdict(list)
        for row in data:
            region_orders[row['Region']].append(int(row['Quantity']) *
float(row['Price']))
        avg_orders = {r: sum(v)/len(v) for r, v in region_orders.items()}
        best_region = max(avg_orders, key=avg_orders.get)

        # 5. Products with zero/negative stock (simulated)
        low_stock = [row['Product'] for row in data if int(row['Quantity']) <= 0]

        # Write report
        with open('analysis_report.txt', 'w') as f:
            f.write("Sales Analysis Report\n")
            f.write("=" * 40 + "\n\n")
            f.write("1. Total Sales by Region:\n")
            for region, total in region_sales.items():
                f.write(f"   {region}: Rs. {total:.2f}\n")
            f.write(f"\n2. Best-Selling Product: {best_product}\n")
            f.write(f"\n3. Daily Sales:\n")
            for date, total in sorted(daily_sales.items()):
                f.write(f"   {date}: Rs. {total:.2f}\n")
            f.write(f"\n4. Highest Avg Order Value: {best_region} (Rs.
{avg_orders[best_region]:.2f})\n")
            f.write(f"\n5. Low Stock Products: {low_stock if low_stock else
'None'}\n")

        # Display results
        print("Analysis complete. Report written to analysis_report.txt")
        print(f"\nRegion Sales: {dict(region_sales)}")
        print(f"Best Product: {best_product}")
        print(f"Best Region: {best_region}")

    except FileNotFoundError:
        print("Error: sales_data.csv not found!")
    except PermissionError:
        print("Error: No file permission!")
    except Exception as e:
        print(f"Error: {str(e)}")
```

`analyze_sales()`

Marking Criteria:
- CSV reading: 1 mark
- Region sales and product analysis: 1 mark
- Daily trend and average calculation: 1 mark
- Report writing: 1 mark
- Exception handling: 1 mark

Viva Voce (5 marks)

1. **Primary key and composite primary key**
   - **Answer:** Primary key uniquely identifies each row. A table can have composite primary key (multiple columns combined). Example: `PRIMARY KEY(roll, sub_id)` in Marks table - individual values may repeat but combination is unique.
   - **Marking:** 1 mark for explanation.
2. **INNER JOIN vs LEFT JOIN**
   - **Answer:** INNER JOIN returns only matching rows from both tables. LEFT JOIN returns all rows from left table and matching from right (NULLs for non-matches). Use INNER when you need only matched data; use LEFT to keep all records from left table.
   - **Marking:** 1 mark for difference.
3. **GROUP BY vs ORDER BY**
   - **Answer:** GROUP BY groups rows with same values for aggregate functions (COUNT, SUM, AVG). ORDER BY sorts the result set by specified column(s). GROUP BY is for aggregation; ORDER BY is for sorting output.
   - **Marking:** 1 mark for explanation.
4. **sqlite3 module and commit()**
   - **Answer:** `sqlite3.connect()` creates connection, `cursor()` creates cursor for executing queries. `commit()` saves changes to database. Without commit(), changes are lost when connection closes (they exist only in memory).
   - **Marking:** 1 mark for explanation.
5. **Normalization 1NF and 2NF**
   - **Answer:** 1NF: Each column contains atomic values, no repeating groups. Violation: phone numbers in comma-separated string. 2NF: 1NF + no partial dependencies (all non-key columns depend on entire primary key). Violation: student name depends only on roll, not on (roll, subject) composite key.
   - **Marking:** 1 mark for each NF with example.